

Formalising Binary Linear Constraint System Games in Lean 4

Research Project at QCC

Sean Perazzolo

EPFL

sean.perazzolo@epfl.ch

June 2026

Abstract

This project presents a Lean 4 formalisation of binary Linear Constraint System (LCS) games and their quantum strategies. It provides the core definitions for LCS games over F_2 , together with two complementary quantum strategy formalisms: a projector-based formalism, where strategies are described by projective measurement systems for the players, and an observable-based formalism, where strategies are described by self-adjoint involutive operators satisfying the relevant commutation relations. The development also includes a bridge between these two formalisms, allowing strategies to be translated from one description to the other.

On the analytic side, the formalisation develops local winning and local loss operators for binary LCS games and proves a sum-of-squares decomposition of the local loss operator. This is then combined with an EPR-state argument to extract row identities for observables arising from bipartite strategies. On the algebraic side, the project defines the solution group of a binary linear system and shows how these row identities induce a matrix representation of the corresponding solution group.

The Mermin-Peres Magic Square game serves as the main case study, illustrating how the abstract framework can be instantiated by an explicit quantum strategy.

Overall, the project provides a machine-checked Lean 4 development connecting binary LCS games, quantum strategies, and solution group representations.

Keywords: Lean 4, Formal verification, Linear Constraint System Games, Quantum strategies, Solution groups, Mermin-Peres magic square game

1 Introduction

1.1 Quantum Non-Local Games

In quantum information theory, non-local games are a standard formalism for studying quantum entanglement and non-locality. Non-locality refers to the ability of quantum systems to exhibit correlations that cannot be explained by local hidden-variable models, or equivalently by the principle of local realism. In a standard non-local game, two or more cooperating players are physically separated and

forbidden from communicating once the game begins. They receive inputs from a referee and must produce outputs satisfying a prescribed winning condition. While classical strategies for non-local games are limited to correlations achievable by shared randomness, quantum strategies can leverage entanglement to achieve higher winning probabilities, often surpassing classical limits. This setting makes it possible to compare the correlations achievable by classical and quantum resources, providing a precise way to study the difference between the two.

1.2 Historical Context and Significance

The study of such games is rooted in the foundations of quantum mechanics. The Einstein-Podolsky-Rosen (EPR) paradox, proposed in 1935 [1], challenged the completeness of quantum theory by arguing that its predictions suggested an unacceptable form of long-range influence, famously described as “spooky action at a distance.” In 1964, Bell’s theorem showed [2] that no local hidden-variable theory can reproduce all quantum predictions, establishing non-locality as a central feature of quantum theory rather than a purely philosophical curiosity. Since then, non-local games have become a standard language for expressing and analysing this phenomenon. They also play an important role in quantum information theory, for instance in device-independent quantum cryptography, where security guarantees are derived from the violation of classical bounds without relying on assumptions about the internal structure of the devices used.

1.3 Linear Constraint System Games

Within this broad class, Linear Constraint System (LCS) games [3], [4] form a particularly simple and structured family. Instead of arbitrary input-output rules, the winning conditions in an LCS game are determined by a system of linear equations over a finite field, most often in the binary setting over F_2 . In such a game, the first player, conventionally called Alice, is asked for an assignment to the variables appearing in a given equation, while the second player, Bob, is asked for the value of a single variable appearing in that equation. The players win if Alice’s assignment satisfies the chosen equation and if Bob’s answer agrees with Alice’s value on the queried variable. Because their underlying

structure is rooted in linear algebra and group theory, LCS games offer a mathematically elegant setting in which to study quantum advantage.

1.3.1 Mermin-Peres Magic Square Game

Canonical examples such as the Mermin-Peres magic square game illustrate the strength of this framework particularly well: they exhibit quantum pseudotelepathy, a phenomenon in which players sharing entanglement can satisfy the constraints with certainty even though no classical strategy can win perfectly.

1.4 Contributions and Scope

This project makes the following contributions to the formalisation of binary Linear Constraint System games in Lean 4:

- It formalises the core definitions of binary Linear Constraint System games, including layouts, assignments, games, and explicit binary linear systems over F_2 .
- It develops two quantum strategy formalisms: a projector-based formalism using projective measurement systems, and an observable-based formalism using self-adjoint involutive operators.
- It implements a bridge between these two formalisms, allowing observable strategies to be translated into projector strategies.
- It defines local and global winning/loss operators for binary LCS games and proves a sum-of-squares decomposition of the local loss operator, which is then used in the EPR-state argument.
- It formalises an EPR-state argument that extracts row identities from local-loss annihilation in the bipartite setting.
- It defines the solution group of a binary linear system and constructs matrix representations of this group from the previously derived row identities.
- It instantiates the general framework on the Mermin-Peres Magic Square game as the main case study.

1.4.1 Repository and Documentation

Source

The project source code is publicly available on GitHub, accessible via

sean.perazzolo.ch/lcs/source.

Documentation

In addition, to make the project easier to navigate, API documentation is available at

sean.perazzolo.ch/lcs/documentation.

Report

A copy of this report is hosted on the project website at sean.perazzolo.ch/lcs/report.

2 Approach

2.1 Structure of the Lean Development

The formalisation is organised as follows:

- The foundational definitions are introduced in `LCS/Basic.lean`, which defines layouts, games, and explicit binary linear systems.
- The strategy layer is developed in `LCS/Strategy`, with separate modules for projector-based strategies, observable-based strategies, and the bridge between them.
- The main proof-oriented part of the project is then divided into several components.
 - `LCS/WinningCondition.lean` defines the winning and loss operators and proves the sum-of-squares decomposition of the local loss operator.
 - `LCS/EPR.lean` extracts matrix identities from local-loss annihilation on the EPR state.
 - `LCS/SolutionGroup.lean` together with `LCS/SolutionGroup/Representation.lean` define the binary solution group and construct its matrix representations.
- Finally, the abstract framework is instantiated in `LCS/Games/MagicSquare`, which develops the Mermin-Peres Magic Square game as the main case study.

2.2 Building the Documentation

This project documentation is built with [doc-gen4](#), the standard Lean documentation tool. It processes the doc-comments attached to lemmas and sections, rendering the mathematical content while hiding the proof details. This is the same tool used for the official mathlib documentation.

2.3 Linear Constraint System Games Formalisation

We begin by formalising Linear Constraint System games.

2.3.1 The Mathematical Object

A linear constraint system over a field K consists of a finite family of variables $x_1, \dots, x_s$ together with a finite family of r equations

$$\sum_{j=1}^s A_{ij} x_j = b_i \quad (1)$$

, where $A_{ij}, b_i \in K$.

In this project, we restrict to the binary setting over F_2 . In this case, variables take values in $\{0, 1\}$ and the equations are evaluated modulo 2.

2.3.2 The Game

In the associated game, the referee selects an equation i and sends it to Alice, while Bob receives a variable j that appears in that equation.

Alice responds with an assignment of values to the variables appearing in the chosen equation, while Bob responds with a value for the queried variable.

The players win if Alice's assignment satisfies the chosen equation and if Bob's answer agrees with Alice's value on the queried variable.

2.3.3 Representation in Lean

We define an `LCSLayout` structure to represent the following data:

- the number of variables s ,
- the number of equations r ,
- the support of each equation, as a family of finite sets $V : \text{Fin } r \rightarrow \text{Finset } (\text{Fin } s)$.

This support-based presentation records, for each equation, the set of variables that occur in it. In the binary setting, this amounts to taking the coefficients to be implicitly equal to 1 on the support of the equation, which is why the layout can be described just by these finite sets.

This structure does not capture the constants b_i on the right-hand side of the equations, since many constructions depend only on the incidence pattern of the variables in each equation.

```
1 structure LCSLayout where
2   r : ℕ
3   s : ℕ
4   V : Fin r → Finset (Fin s)
```

Next, we define an `LCSGame` structure that extends `LCSLayout` by including the constants $b : \text{Fin } G.r \rightarrow \text{Fin } 2$, which represent the right-hand side of the equations in the binary setting.

```
1 structure LCSGame (G : LCSLayout) where
2   b : Fin G.r → Fin 2
```

Finally, for the group-theoretic constructions, we also define a `LinearSystem` structure as an alternative description of a binary LCS game. This structure consists of a coefficient matrix A and a right-hand side vector b .

```
1 structure LinearSystem where
2   layout : LCSLayout
3   A : Fin layout.r → Fin layout.s → Fin 2
4   b : Fin layout.r → Fin 2
```

Any support-based game can be converted into such a system by taking $A_{ij} = 1$ when variable j appears in equation i , and $A_{ij} = 0$ otherwise.

The project therefore uses both a support-based description, via `LCSLayout` and `LCSGame`, and a matrix-based description, via `LinearSystem`, depending on which is more convenient for the construction at hand.

For a concrete example of these definitions, see the case study of the Mermin-Peres Magic Square game in Section 4.

2.4 Quantum Strategy Formalisms

2.4.1 Projector-Based Strategies

The Mathematical Object In a quantum strategy, a player's response to a question is not modelled as a deterministic function from questions to answers. Instead, the question determines which measurement the player performs on their share of quantum state, and the answer is then

given by the outcome of that measurement. For this reason strategies are naturally described in terms of measurement systems, which are families of projective measurements.

In the binary LCS setting, Alice and Bob have different answer types. When Alice is asked an equation i , she must provide a full assignment to all variables appearing in that equation. In Lean, an assignment for equation i is defined simply as a function mapping each variable in V_i to a binary value:

```
1 def Assignment (G : LCSLayout) (i : Fin G.r) :
2   Type :=
3   (j : G.V i) → Fin 2
```

Her measurement is therefore indexed by the set of assignments on that equation. When Bob is asked a variable j , he must provide a single bit, so his measurement is indexed by the two outcomes in F_2 .

The project works with projective measurements. This means that for each question, the possible answers are indexed by a family of orthogonal self-adjoint idempotent operators summing to the identity, which represent the projectors onto the corresponding outcome subspaces.

Representation in Lean

In Lean, projective measurements are encoded by the predicate `IsMeasurementSystem`. For a finite family of operators $f : I \rightarrow R$, this predicate expresses that the operators form a projective measurement: They sum to the identity, are self-adjoint, are idempotent, and are pairwise orthogonal.

```
1 structure IsMeasurementSystem
2   {I : Type*} [Fintype I]
3   (f : I → R) : Prop where
4   sum_one      : ∑ x, f x = 1
5   idempotent    : ∀ x, f x * f x = f x
6   orthogonal    : ∀ x y, x ≠ y → f x * f y = 0
7   self_adjoint : ∀ x, star (f x) = f x
```

Using this notion, the projector-based strategy formalism is defined by the structure `ProjectorStrategy`.

```
1 structure ProjectorStrategy
2   (R : Type*) [Ring R] [StarRing R] [Algebra ℂ R]
3   (G : LCSLayout) where
4   E : ∀ i, (Assignment G i → R)
5   F : Fin G.s → (Fin 2 → R)
6   alice_ms : ∀ i, IsMeasurementSystem (E i)
7   bob_ms    : ∀ j, IsMeasurementSystem (F j)
8   commute   : ∀ i j α β, E i α * F j β = F j β * E i α
```

Here $E\ i$ denotes Alice's projective measurement associated with equation i ; it is the full family of operators indexed by all assignments $x : \text{Assignment } G\ i$. For a specific assignment x , the operator $E\ i\ x$ is the projector onto the event that Alice answers exactly x when asked equation i . Similarly, $F\ j$ denotes Bob's binary projective measurement associated with variable j , and for a bit $y : \text{Fin } 2$, the operator $F\ j\ y$ is the projector onto the event that Bob answers y when asked variable j . The fields `alice_ms` and `bob_ms` assert that these families define projective measurements. Finally, the commutation condition expresses the

operator-theoretic separation between Alice and Bob: Alice’s and Bob’s measurement operators commute for all questions and outcomes.

Although this formalism is expressed in terms of projective measurements, the later development also makes systematic use of observables derived from these measurements. Their role is explained after the observable-based formalism has been introduced.

2.4.2 Observable-Based Strategies

Although the projector-based formalism is the most direct way to describe quantum strategies, it is often more convenient to work with observables, which are self-adjoint operators whose spectral decomposition corresponds to the projective measurements. For this reason, the project also introduces an observable-based formalism.

In this setting, a strategy is described by one observable for each variable on Alice’s side, and one observable for each variable on Bob’s side. They must satisfy the commutation relation dictated by the structure of the game. On Alice’s side, observables corresponding to variables appearing in the same equation must commute, so that their products are well defined independently of the order of multiplication. In addition Alice’s observables must commute with all of Bob’s observables, reflecting the spatial separation between the players.

Because the present project is restricted to binary outcomes, the observable formalism is also specialised accordingly. Rather than considering arbitrary observables, we work with self-adjoint involutions, which are exactly the operators arising from two-outcome projective measurements. This is encoded in Lean by the predicate `IsObservable`.

```
1 structure IsObservable (O : R) : Prop where
2   involutive   : O * O = 1
3   self_adjoint : star O = O
```

With this notion of binary observable in place, the project packages the observable description of a strategy into a structure `ObservableStrategy` :

```
1 structure ObservableStrategy
2   (R : Type*) [Ring R] [StarRing R] [Algebra ℂ R]
3   [StarModule ℂ R]
4   (G : LCLayout) where
5     alice_obs : Fin G.s → R
6     bob_obs   : Fin G.s → R
7     alice_observable : ∀ j, IsObservable (alice_obs j)
8     bob_observable   : ∀ j, IsObservable (bob_obs j)
9     sameEquation_comm :
10      ∀ i, Pairwise (fun j k : G.V i ⇒ Commute
11        (alice_obs j.1) (alice_obs k.1))
12      alice_bob_commute :
13      ∀ j k, Commute (alice_obs j) (bob_obs k)
```

2.4.3 Bridge Between Projector and Observable-Based Strategies

The two formalisms are closely related, and it is possible to translate strategies between these two descriptions. This bridge is essential for this project as explicit examples are

most often described in terms of observables, while the main developments, such as the loss operator, are more naturally expressed in terms of projective measurements. Note, however, that the provided translation is strictly a one-way construction from observables to projectors, rather than a canonical two-way equivalence. The reason for this asymmetry will become clear once the mechanism for extracting observables from projectors is introduced below.

On Bob’s side, the passage from projectors to observables is immediate. Since Bob’s measurements are binary, each family $F_j : \text{Fin } 2 \rightarrow R$ gives rise to a single observable obtained from the difference of the two projectors. In Lean, this is the definition `Bob_B`.

Alice’s side is slightly subtler. For a fixed equation i , the family E_i is indexed by full assignments rather than by binary outcomes. To extract an observable associated with a single variable j appearing in that equation, one first collapses the assignment-indexed measurement to a binary measurement that only distinguishes the value of the variable j . This is expressed in Lean by the construction `InducedMeasurementSystem (strat.E i) (fun x ⇒ x j)`. The observable associated with this induced binary measurement is then defined as `Alice_A strat i j`.

```
1 def ObservableOfMeasurementSystem (f : Fin 2 → R) :
2   R :=
3     f 0 - f 1
4
5 def Alice_A
6   (strat : ProjectorStrategy R G) (i : Fin G.r) (j :
7     G.V i) : R :=
8     ObservableOfMeasurementSystem
9     (InducedMeasurementSystem (strat.E i) (fun x ⇒ x j))
10
11 def Bob_B (strat : ProjectorStrategy R G) (j : Fin
12   G.s) : R :=
13     ObservableOfMeasurementSystem (strat.F j)
```

The project also proves that these derived operators are genuine binary observables. Bob’s observable is obtained directly from his binary measurement, while Alice’s observable is obtained from the induced binary measurement associated with a single variable in a fixed equation. In both cases, the fact that the underlying family is a measurement system implies that the resulting operator is a self-adjoint involution.

It is now possible to see why the formalisms do not perfectly round-trip. While observables can be extracted from a projector strategy as just described, those derived observables on Alice’s side (`Alice_A`) are intrinsically indexed by a pair (i, j) of an equation and a variable within it. In contrast, an `ObservableStrategy` demands a single global observable A_j for each variable j .

Conversely, starting from an observable-based strategy, the project constructs a projector-based strategy by taking the two spectral projectors associated with each observable. Bob’s measurement is obtained directly from his observable, while Alice’s measurement is built by combining the projectors associated with the commuting observables appearing in a common equation. This construction is implemented in Lean by `ObservableStrategy_To_ProjectorStrategy`.

```

1 noncomputable def
  ObservableStrategy_To_ProjectorStrategy
2 {R : Type*} [Ring R] [StarRing R] [Algebra C R]
  [StarModule C R]
3 {G : LCSLayout}
4 (S : ObservableStrategy R G)
5 :
6   ProjectorStrategy R G := {
7     E := AliceMeasurementFromObservables S
8     F := BobMeasurementFromObservables S
9     alice_ms :=
10    aliceMeasurementFromObservables_isMeasurementSystem S
11    bob_ms :=
12    bobMeasurementFromObservables_isMeasurementSystem S
13    commute := aliceMeasurement_bobMeasurement_commute S
14  }

```

On Bob's side, each observable B_j gives a binary measurement by taking its two associated spectral projectors, defined by the formula $F_{j,y} = \frac{1}{2}(I + (-1)^y B_j)$ for outcomes $y \in \{0, 1\}$, and the corresponding family is proved to form a measurement system. On Alice's side, the measurement projector $E_{i,x}$ attached to an equation i and an assignment x is obtained by multiplying the individual variable projectors associated with the observables appearing in that equation:

$$E_{i,x} = \prod_{j \in V_i} \frac{1}{2}(I + (-1)^{x_j} A_j) \quad (2)$$

The row-wise commutation assumptions ensure that these products are well defined independently of the multiplication order, and the project proves that the resulting family is again a measurement system. Finally, the global commutation hypothesis between Alice's and Bob's observables is used to show that every Alice projector commutes with every Bob projector. These results together justify the construction of `ObservableStrategy_To_ProjectorStrategy` as a valid `ProjectorStrategy`.

2.4.4 Bipartite Observable Strategies

For the final part of the project, the observable formalism is further specialised to a bipartite setting. This is the setting relevant for the EPR-state argument developed later, where Alice's and Bob's operators act on different tensor factors of a bipartite Hilbert space.

Instead of specifying two separate observable families from the start, the project begins with a single family of observables indexed by the variables of the game. These observables act on a space of the form $\text{Matrix } n \ n \ \mathbb{C}$. From this single family, one obtains Alice's and Bob's observables by lifting them to the two tensor factors: Alice's observable associated with a matrix M is $M \otimes I$, while Bob's observable is $I \otimes M$. In this way, commutation between Alice's and Bob's operators is automatic, since operators acting on different tensor factors commute.

In Lean, this data is encoded by the structure `BipartiteObservableStrategy`.

```

1 structure BipartiteObservableStrategy
2 (n : Type*) [Fintype n] [DecidableEq n]
3 (G : LCSLayout) where
4   obs : Fin G.s → Matrix n n C
5   is_observable : ∀ j, IsObservable (obs j)
6   sameEquation_comm :
7     ∀ i, Pairwise (fun j k : G.V i ⇒ Commute (obs
8       j.1) (obs k.1))

```

Here `obs j` denotes the basic observable associated with variable j . The field `is_observable` asserts that each of these operators is a self-adjoint involution, while `sameEquation_comm` expresses the row-wise commutation required to form products of observables along a common equation.

From such a bipartite observable strategy, the project constructs an ordinary observable strategy by tensor lifting. Alice's observables are obtained by applying the map $M \mapsto M \otimes I$, and Bob's observables by applying $M \mapsto I \otimes M$. This construction is implemented by `toObservableStrategy`, and it provides the entry point from the bipartite setting into the general observable and projector formalisms developed earlier.

This specialisation is important because it matches the tensor-product structure of the EPR state used later in the project. In particular, it provides the framework in which local-loss annihilation on the EPR state can be turned into concrete matrix identities and, ultimately, into representations of the solution group.

2.5 Notation Used in the Sequel

From this point on, several sections use the local notation introduced in the Lean development for the operators attached to a projector-based strategy `strat` and a game `game`. This improves readability by allowing us to write concrete operator identities without having to refer to the underlying strategy and game structures at every step. The notation is as follows:

```

1 local notation "A["i", "j"]" ⇒ Alice_A strat i j
2 local notation "B["j"]" ⇒ Bob_B strat j
3 local notation "E["i", "x"]" ⇒ strat.E i x
4 local notation "F["j", "y"]" ⇒ strat.F j y
5 local notation "b["i"]" ⇒ game.b i
6 local notation "⌈["i"]" ⇒ Alice_Row_Prod strat i

```

- $A[i, j]$ denotes the derived Alice observable `Alice_A strat i j`, attached to equation i and to a variable j appearing in that equation.
- $B[j]$ denotes the derived Bob observable `Bob_B strat j`, attached to variable j .
- $E[i, x]$ denotes the projector `strat.E i x` corresponding to Alice answering the assignment x on equation i .
- $F[j, y]$ denotes the projector `strat.F j y` corresponding to Bob answering the bit y on variable j .
- $b[i]$ denotes the right-hand side bit `game.b i` of equation i .

We also use the notation $\lceil[i]$ for the product of Alice's derived observables along row i , implemented in Lean as `Alice_Row_Prod strat i`. Concretely, this reproduces the row product

$$\prod_{k \in V_i} A_k^{(i)} \quad (3)$$

, that is, the product of the observables attached to all variables appearing in equation i .

2.6 Winning Conditions and Local Loss

Once a binary LCS game and a projector-based strategy have been defined, the next step is to formalise the winning condition of the game and to derive the associated winning and loss operators. This is the point at which the right-hand side values b_i of the equations come into play, as the winning condition depends on whether an assignment satisfies the chosen constraint.

2.6.1 Winning and Loss Operators

For a fixed equation i , the set of winning assignments consists of the assignments whose parity matches b_i . This set is used to define the local winning operator for a pair (i, j) of an equation and a variable appearing in that equation. Intuitively, this operator collects exactly those outcomes for which Alice's assignment satisfies the equation and agrees with Bob's answer on variable j .

In the codebase, the notation $S[i]$ is used as a shorthand for $\text{winning_assignments game } i$, namely the set of assignments on equation i whose parity matches b_i .

```
1 def winning_assignments (i : Fin G.r) : Finset
  (Assignment G i) :=
2   Finset.univ.filter (fun α => (Σ j : G.V i, (α j :
  Fin 2)) = b[i])
3
4 noncomputable def local_winning_operator (i : Fin
  G.r) (j : G.V i) : R :=
5   Σ x ∈ S[i], E[i, x] * F[j, x j]
6
7 noncomputable def local_loss_operator (i : Fin G.r)
  (j : G.V i) : R :=
8   1 - local_winning_operator game strat i j
```

The local loss operator is defined as the complement of the local winning operator. It is the central object in the analytic part of the project, since the sum-of-square decomposition is proved for this operator.

In addition to these local quantities, the project also defines the global winning and loss operators by averaging the local ones.

```
1 noncomputable def winning_operator : R :=
2   Σ i : Fin G.r, Σ j : G.V i,
3   let normalization : ℂ := (G.r * (G.V i).card : ℕ)
4   (1 / normalization) * local_winning_operator game
  strat i j
5
6 noncomputable def loss_operator : R :=
7   1 - winning_operator game strat
```

2.7 EPR Extraction Framework

2.7.1 The EPR Vector

At this point, the formalisation is specialised from the earlier abstract algebraic setting to finite-dimensional complex matrix algebras. More precisely, the relevant

operators act on a bipartite space of the form $\mathbb{C}^n \otimes \mathbb{C}^n$, represented in Lean by matrices of type $\text{Matrix } (n \times n) \ (n \times n) \ \mathbb{C}$.

The distinguished vector used in the project is the unnormalised maximally entangled vector.

$$|\Omega\rangle = \sum_a e_a \otimes e_a \quad (4)$$

Its importance lies in the symmetry with which it couples the two tensor factors: it allows operators acting on one side of the tensor product to be related to operators acting on the other side:

$$(M \otimes I)|\Omega\rangle = (I \otimes M^T)|\Omega\rangle \quad (5)$$

In Lean, the EPR vector is defined as follows.

```
1 noncomputable def eprVec
2   (n : Type*) [Fintype n] [DecidableEq n] : (n × n)
3   → ℂ :=
4   fun ab => if ab.1 = ab.2 then 1 else 0
5   local notation "Ω" => eprVec n
```

This is simply the coordinate description of the vector $|\Omega\rangle$, written in the standard basis of the bipartite space. The vector is intentionally left unnormalised, since the later arguments only use annihilation and injectivity properties rather than norm considerations.

2.7.2 EPR Identities for Bipartite Operators

The key role of the EPR vector is that it turns relations on the bipartite space into ordinary matrix identities. Concretely, if M and N are complex matrices, then the action of $M \otimes N$ on $|\Omega\rangle$ can be computed explicitly, and vanishing of this action is equivalent to a matrix equation involving M and N^T .

The fundamental identity proved in the project is that $(M \otimes N)|\Omega\rangle = 0$ if and only if $MN^T = 0$. This gives the basic extraction principle used later in the development.

```
1 lemma kronecker_mulVec_epr_eq_zero_iff
2   (n : Type*) [Fintype n] [DecidableEq n]
3   (M N : Matrix n n ℂ) :
4   Matrix.mulVec (M ⊗_k N) (eprVec n) = 0 ↔
5   M * N^T = 0
```

The project also proves the corresponding affine variant, which is the form used when extracting identities from operators of the form $1 - M \otimes N$.

```
1 lemma one_sub_kronecker_mulVec_epr_eq_zero_iff
2   (n : Type*) [Fintype n] [DecidableEq n]
3   (M N : Matrix n n ℂ) :
4   Matrix.mulVec (1 - M ⊗_k N) (eprVec n) = 0 ↔
5   M * N^T = 1
```

Several specialised versions of this identity are then derived for the bipartite lift operations introduced earlier. These lemmas make it possible to replace operator equalities on the distinguished vector $|\Omega\rangle$ by concrete matrix equalities in the underlying $n \times n$ space.

This is the mechanism referred to in the project as the EPR extraction argument, and it forms the bridge between bipartite operator relations and the matrix identities used later in the representation-theoretic part of the development.

2.8 Defining the Solution Group of a Binary Linear System

The final algebraic object introduced in the project is the solution group associated with a binary linear system. In the binary LCS setting, this group packages the combinatorial structure of the constraints into a presented group whose generators correspond to variables and whose relations encode the equations of the system.

More precisely, let

$$\sum_{j=1}^s A_{ij} x_j = b_i \quad (6)$$

be a binary linear system over F_2 . Its solution group is generated by symbols $g_1, \dots, g_s$ together with a distinguished central element J , subject to the following relations:

- each generator g_j is an involution,
- J is an involution,
- J commutes with every g_j ,
- whenever two variables appear in a common equation, the corresponding generators commute,
- for each equation i , the product of the generators corresponding to the variables appearing in that equation is equal to $J^{(b_i)}$.

This construction is specific to the binary setting. The involutive nature of the generators reflects the fact that the observables appearing earlier in the project are ± 1 -valued, and the row relation records the parity condition associated with each equation.

In Lean, the solution group is defined from the structure `LinearSystem`, which provides both the coefficient matrix A and the right-hand side vector b . The generators are represented by an inductive type containing one constructor for the variable generators and one for the distinguished element J . The relations are then imposed by passing to the presented group generated by these symbols.

At this stage, the project works with `LinearSystem` rather than directly with `LCSGame`. This is because the solution-group presentation is most naturally phrased in terms of explicit coefficients and equation supports extracted from a coefficient matrix. In particular, the commuting and row-product relations are defined by reading off which variables occur in each equation from the matrix A , while the right-hand side vector b determines the exponent of the distinguished generator J .

```

1  inductive SolutionGen (S : LinearSystem) where
2    | var : Fin S.layout.s → SolutionGen S
3    | J : SolutionGen S
4
5  def solutionRelators (S : LinearSystem) : Set
6    (FreeGroup (SolutionGen S)) :=
7    fun w =>
8      (∃ j, w = involutionRel (genVar (S := S) j)) v
9      w = involutionRel (genJ (S := S)) v
10     (∃ j, w = commuteRel (genVar (S := S) j) (genJ
11       (S := S))) v
12     (∃ j k, j < k ∧ sameEquation S j k ∧
13       w = commuteRel (genVar (S := S) j) (genVar
14         (S := S) k)) v
15     (∃ i, w = equationRelator S i)
16
17 abbrev SolutionGroup (S : LinearSystem) : Type :=
18   PresentedGroup (solutionRelators S)

```

Thus the solution group is defined in Lean as the presented group on the generators `SolutionGen S`, quotiented by the set of relators `solutionRelators S` encoding the involution, commutation, centrality, and row-product relations. The project therefore treats the solution group as an explicitly presented algebraic object attached to a binary linear system. This group will later serve as the target of the representation-theoretic part of the development, where matrix identities extracted from the EPR argument are used to verify its defining relations.

3 Results

With the definitions and constructions described in the previous section, we can now formalise the results of interest, which follow the development in Arthur Mehta's thesis [5].

3.1 Sum-of-Squares Decomposition

The first main result is a sum-of-squares decomposition of the local loss operator. The proof in mathematical terms is described in section 4.7 of Mehta's thesis [5].

$$\begin{aligned}
 L_{i,j} &= 1 - \sum_{x,y: x \in S[i], y=x_j} E_{i,x} F_{j,y} \\
 &= \frac{1}{8} \left(\left(I - B_j A_j^{(i)} \right)^2 \right. \\
 &\quad \left. + \left(I - (-1)^{b_i} \prod_{k \in V_i} A_k^{(i)} \right)^2 \right. \\
 &\quad \left. + \left(I - (-1)^{b_i} \prod_{k \in V_i} A_k^{(i)} A_j^{(i)} B_j \right)^2 \right) \quad (7)
 \end{aligned}$$

Hence, the local loss vanishes if and only if each squared term vanishes individually, since each is positive semidefinite. This decomposition is a key step in the EPR extraction argument to produce the row identities.

In Lean, this decomposition is formalised by the following theorem:

```

1 theorem local_loss_sos (i : Fin G.r) (j : G.V i) :
2   local_loss_operator game strat i j =
3     (1/8 : ℂ) • (
4       (1 - B[j] * A[i, j])^2 +
5       (1 - (-1 : ℂ)^(b[i]).val • Π[i])^2 +
6       (1 - (-1 : ℂ)^(b[i]).val • (Π[i] * A[i, j] *
7         B[j]))^2

```

This formalises that given a game and a projector-based strategy for it, the local loss operator can be decomposed as a sum of three squares.

The main challenges in the formalisation of this result were:

- 1) Noncommutative operator algebra. While the proof is mathematically elementary, Lean needs to be explicit about where multiplication is noncommutative and where it is safe to reorder terms. A lot of the work is proving and reusing commutation lemmas like :
 - Alice observables commute along a row.
 - Alice and Bob operators commute when needed.
 - The Row product commutes with relevant local observable.
- 2) Mixing scalar actions with operator multiplication. A repeated source of complication was expressions involving both scalar multiplication and operator multiplication ($c \cdot X$), ($X * Y$). The paper treats these transparently, but in Lean they require careful rewriting with lemmas like `smul_mul` and `mul_smul` to put the scalar factors in the right place.

- 3) Turning the paper sums into explicit finite sums in Lean. The proof uses sums over winning assignments and marginal slices of assignments. In Lean these are implemented using `Finset.filter`, `Finset.sum_congr` and fiberwise sums. Consequently, a significant part of the file is showing that the paper sums can be rewritten in terms of these more explicit constructions, and then manipulating them to get the desired final form.

3.2 Row Identities Extraction

The next step is to show that local-loss annihilation on the EPR state implies three local relations, which can then be extracted into identities on the underlying matrix space.

This is a two-step process.

- 1) First, using the sum-of-squares decomposition, we show that if the local loss operator annihilates the EPR state, then each SOS term annihilates $|\Omega\rangle$ individually. Writing

$$\begin{aligned}
 T_1 &= I - B_j A_j^{(i)}, \\
 T_2 &= I - (-1)^{b_i} \prod_{k \in V_i} A_k^{(i)}, \\
 T_3 &= I - (-1)^{b_i} \prod_{k \in V_i} A_k^{(i)} A_j^{(i)} B_j
 \end{aligned} \quad (8)$$

From the SOS decomposition, we get :

$$L_{i,j}|\Omega\rangle = 0 \implies \frac{1}{8}(T_1^2 + T_2^2 + T_3^2)|\Omega\rangle = 0, \quad (9)$$

Each T_k is self-adjoint: this follows from the fact that the local Alice and Bob observables are self-adjoint, that the Alice observables appearing in the same row commute so that their product is again self-adjoint, and that the scalar factor $(-1)^{b_i}$ is real. Taking the Hermitian inner product with $|\Omega\rangle$ gives

$$\begin{aligned}
 \langle \Omega | (T_1^2 + T_2^2 + T_3^2) \Omega \rangle &= \langle T_1 \Omega | T_1 \Omega \rangle \\
 &\quad + \langle T_2 \Omega | T_2 \Omega \rangle \\
 &\quad + \langle T_3 \Omega | T_3 \Omega \rangle.
 \end{aligned} \quad (10)$$

Each summand is a norm square, hence a nonnegative real number. Since their sum is zero, each one must itself be zero, and therefore

$$T_1|\Omega\rangle = 0, \quad T_2|\Omega\rangle = 0, \quad T_3|\Omega\rangle = 0. \quad (11)$$

This is the positivity argument formalised in Lean.

```

1 lemma three_selfAdjoint_squares_mulVec_eq_zero
2   (hT1 : T1.H = T1) (hT2 : T2.H = T2) (hT3 : T3.H =
3     T3)
4   (h :
5     Matrix.mulVec (T1 ^ 2 + T2 ^ 2 + T3 ^ 2) v =
6     0) :
7     Matrix.mulVec T1 v = 0 ∧ Matrix.mulVec T2 v = 0 ∧
8     Matrix.mulVec T3 v = 0

```

- 2) Then, using the extraction lemmas introduced in the EPR framework, each annihilation relation is rewritten in bipartite form and converted into an ordinary matrix identity. These lemmas are applied to the three

SOS terms after expressing Alice's operators as lifts $A \otimes I$ and Bob's operators as lifts $I \otimes B$.

For example, for the first term, one rewrites the consistency operator using $(I \otimes B_j)(A_j^{(i)} \otimes I) = A_j^{(i)} \otimes B_j$:

$$\begin{aligned} T_1|\Omega\rangle = 0 &\implies (I - A_j^{(i)} \otimes B_j)|\Omega\rangle = 0 \\ &\implies A_j^{(i)}(B_j)^T = I. \end{aligned} \quad (12)$$

Since B_j is an observable, it is an involution, and right-multiplying by $(B_j)^T$ yields:

$$A_j^{(i)} = (B_j)^T. \quad (13)$$

In Lean, these three extraction steps are packaged as separate lemmas, for the above relation the statement is as follows:

```
1  lemma consistency_of_epr_annihilates
2    (i : Fin G.r) (j : G.V i)
3    (A B : Matrix n n ℂ)
4    (hCons :
5      Matrix.mulVec
6        (sosConsistencyTerm strat i j)
7        Ω = 0)
8    (hAlice : Alice_A strat i j = bipartiteAliceLift
9  A)
10   (hBob : Bob_B strat ↑j = bipartiteBobLift B)
11   (hBobs : IsObservable B) :
12   A = BT
```

Applying the same mechanism to the other two terms yields the three identities

$$\begin{aligned} A_j^{(i)} &= B_j^T, \\ \prod_{k \in V_i} A_k^{(i)} &= (-1)^{b_i} I, \\ (-1)^{b_i} \prod_{k \in V_i} A_k^{(i)} A_j^{(i)} B_j^T &= I. \end{aligned} \quad (14)$$

Together these give the row identities that are used in the construction of representations of the solution group.

3.3 Matrix Representations of the Solution Group

The final step of this project is to show that these row identities can be used to construct a matrix representation of the solution group of the binary linear system associated with the game, given a perfect quantum strategy for the game (see Section 4.2.3 of [5]).

3.3.1 The Construction

Suppose a bipartite observable strategy with observables $O_1, \dots, O_s$ achieves perfect play, so that the local loss operators annihilate the EPR state for every equation and every support element. By the extraction argument of the previous subsection, this yields the row identities

$$\prod_{j \in \text{supp}(i)} O_j = (-1)^{b_i} I, \quad \forall i. \quad (15)$$

The representation is then defined by mapping the generators of the solution group to concrete unitary matrices:

- each variable generator g_j is sent to the corresponding observable O_j ,
- the distinguished central generator J is sent to the scalar matrix $-I$.

This assignment respects all four families of defining relations:

- 1) *Involutions*. Each observable satisfies $O_j^2 = I$ by definition, and $(-I)^2 = I$.
- 2) *Centrality*. The scalar matrix $-I$ commutes with every matrix.
- 3) *Same-equation commutation*. Observables appearing in a common equation commute, which is a hypothesis of the observable strategy formalism.
- 4) *Equation relators*. The row identity $\prod_{j \in \text{supp}(i)} O_j = (-1)^{b_i} I$ is exactly the relation $\prod g_j = J^{b_i}$ under the assignment $g_j \mapsto O_j$, $J \mapsto -I$.

Since every relator in the presentation maps to the identity under this assignment, the universal property of the presented group guarantees that the assignment extends uniquely to a group homomorphism

3.3.2 Formalisation in Lean

The first step is to define the generator-level assignment. Each solution-group generator is mapped to a concrete element of the unitary group $\text{unitary}(\text{Matrix } n \ n \ \mathbb{C})$: variable generators are sent to their packaged observables, and J is sent to the scalar matrix $-I$.

```
1  noncomputable def solutionGroupGeneratorImage
2    (obs : Fin S.layout.s → Matrix n n ℂ)
3    (obs_is_observable :
4      ∀ j, IsObservable (obs j)) :
5    SolutionGen S → unitary (Matrix n n ℂ)
6  | .var j ⇒
7    observableMatrixUnitary (obs j)
8    (obs_is_observable j)
9  | .J ⇒ negOneMatrixUnitary
```

This map is then lifted to the free group on the generators via `FreeGroup.lift`, and the representation is obtained by showing that every relator in the solution group maps to the identity under this lift. The generic representation constructor takes as input a family of observables, proofs that they satisfy the same-equation commutation requirement, and proofs that each equation relator maps to the identity in the unitary group. It then applies Mathlib's `PresentedGroup.toGroup` to produce the group homomorphism.

```
1  noncomputable def solutionGroupRepresentation
2    (obs : Fin S.layout.s → Matrix n n ℂ)
3    (obs_is_observable : ∀ j, IsObservable (obs j))
4    (hsame : ∀ {j k}, sameEquation S j k
5      → Commute (obs j) (obs k))
6    (hequation : ∀ i, FreeGroup.lift
7      (solutionGroupGeneratorImage obs
8      obs_is_observable)
9      (equationRelator S i) = 1) :
10   SolutionGroup S →* unitary (Matrix n n ℂ) :=
11   PresentedGroup.toGroup (lift_relators_eq_one ...)
```

The hypothesis `hequation` requires that the lift of each equation relator evaluates to the identity in the unitary

group. Internally, the proof dispatches the remaining three relator families (involutions, centrality, and commutation) automatically, since these follow from the observable axioms and the commutation hypothesis.

The project then provides a second, higher-level constructor that chains the entire pipeline from local-loss annihilation on the EPR state. Starting from the hypothesis that the local loss operator annihilates the EPR vector for every equation i and every support element j , the construction first extracts the row identities via `rowObservableProduct_eq_sign_of_local_loss`, and then passes these identities to the generic representation constructor above.

```

1  noncomputable def solutionGroupRepresentationOfEPRLoss
2    (strat : BipartiteObservableStrategy n G)
3    (hNonempty : ∀ i, Nonempty (G.V i))
4    (hLoss : ∀ i (j : G.V i),
5      Matrix.mulVec
6        (local_loss_operator game
7          strat.toProjectorStrategy i j)
8        (eprVec n) = 0) :
9    SolutionGroup game.toLinearSystem
10   → unitary (Matrix n n ℂ)

```

This is the main end-to-end result of the project. It shows that any bipartite observable strategy achieving perfect play on a binary LCS game gives rise to a unitary representation of the associated solution group. The representation lands in the unitary group (rather than just the general linear group) because observables are by definition self-adjoint involutions, and the scalar matrix $-I$ is trivially unitary.

3.3.3 Main Formalisation Challenge

The formalisation of this step is roughly 600 lines long, which may seem surprising given that the mathematical argument is short: define the generator map, check the relators, invoke the universal property. The bulk of the formalisation is devoted to bridging between two different descriptions of the same algebraic object.

On the group-theoretic side, the equation relator for row i is a word in the free group on the generators `SolutionGen S`, constructed from the explicit equation support of the linear system. On the analytic side, the row identity extracted from the EPR argument is a matrix equation involving the ordered product of observables indexed by the game's row support. These two descriptions refer to the same underlying product, but they arise from different data structures: the relator is built from the `LinearSystem`'s coefficient matrix (via `eqSupport` and `equationWord`), while the row observable product is built from the `LCSGame`'s row support (via `orderedSupportProduct`).

The key bridge lemma `lift_equationRelator_of_rowIdentity` closes this gap. It shows that evaluating the free-group lift of the equation relator under the generator map yields the same matrix as the row observable product, so that the matrix identity $\prod_{j \in \text{supp}(i)} O_j = (-1)^{b_i} I$ can be used directly to verify the relator. Proving this requires a chain of intermediate steps: converting the game's support set to the linear system's equation support, showing that the sorted list product agrees

with the `Finset.noncommProd` used in the strategy layer, and carefully tracking the passage between the free-group word evaluation and the matrix product. This kind of alignment work, connecting two representations of the same mathematical object through Lean's type system, accounts for the majority of the file's length.

4 Magic Square Game Case Study

The Mermin-Peres Magic Square game [6] is the main concrete example developed in this project. It is a particularly natural case study for binary LCS games: the rules are simple to state, the contradiction for classical assignments is easy to understand, and the quantum strategy can be written as an explicit 3×3 grid of Pauli observables. For this reason it is often regarded as one of the most intuitive examples of quantum pseudotelepathy.

4.1 The Game

The game is built from a 3×3 array of binary variables. The referee may ask for one of the three rows or one of the three columns, so there are six possible equation questions in total. Alice receives one of these six questions and must provide values for the three cells lying in that row or column. Bob receives one cell contained in Alice's question and must provide the value of that single cell.

The winning condition is the same as for any binary LCS game, and it has two parts. First, Alice's assignment must satisfy the parity rule attached to the row or column she was asked about. Second, Bob's answer must agree with Alice's value on the overlapping cell.

The parity rules of this game are as follows: all three row equations have even parity, the first two column equations have even parity, and the final column has odd parity. Equivalently, if the variables are denoted by

$$x_1, x_2, \dots, x_9 \in F_2, \quad (16)$$

then the six equations are

$$\begin{aligned}
x_1 + x_2 + x_3 &= 0, \\
x_4 + x_5 + x_6 &= 0, \\
x_7 + x_8 + x_9 &= 0, \\
x_1 + x_4 + x_7 &= 0, \\
x_2 + x_5 + x_8 &= 0, \\
x_3 + x_6 + x_9 &= 1.
\end{aligned} \quad (17)$$

This game exhibits the concept of pseudotelepathy. To see why no perfect classical strategy exists, observe first that any deterministic perfect classical strategy would have to define a single global value for each cell of the square. Bob answers one cell at a time, so his strategy fixes a bit for each variable, and perfect consistency forces Alice to use exactly those same values whenever that variable appears in a row or column question. Thus a perfect classical strategy would induce a global assignment satisfying all six parity equations simultaneously.

This is however impossible: if one sums the three row equations over F_2 , each variable appears exactly once and the total right-hand side is 0. If one instead sums the three

column equations, one obtains the same left-hand side, since the same nine variables appear exactly once again, but now the total right-hand side is 1. Hence the same quantity would have to be equal to both 0 and 1, a contradiction. Therefore no deterministic classical strategy can win perfectly, and hence no classical strategy can win with certainty. Nevertheless, quantum players sharing entanglement can satisfy the game conditions perfectly.

4.2 The Observable Grid

The standard quantum strategy for the magic square is given by a 3×3 grid of commuting two-qubit observables:

$$\begin{pmatrix} X \otimes I & I \otimes X & X \otimes X \\ I \otimes Y & Y \otimes I & Y \otimes Y \\ X \otimes Y & Y \otimes X & Z \otimes Z \end{pmatrix}. \quad (18)$$

Each entry is a self-adjoint involution, hence a binary observable with eigenvalues ± 1 . The crucial structural facts are:

- the three observables in each row commute pairwise,
- the three observables in each column commute pairwise,
- the product along each row is I ,
- the product along the first two columns is I ,
- the product along the final column is $-I$.

These identities match the parity pattern of the game exactly. Because the observables in each row or column commute, they can be measured simultaneously. Moreover, the product constraint has the correct sign in each case: a row or column whose parity bit is 0 has product I , while the final column, whose parity bit is 1, has product $-I$. In this way, the classical parity equations are replaced by operator identities with the same sign pattern.

4.3 The Support-Based Description in Lean

In the formalisation, the game is first encoded in the support-based language introduced earlier. The layout records only which variables occur in each equation, while the right-hand side vector records the parity bits.

For the magic square, the layout has 6 equations and 9 variables. The support function lists the three cells occurring in each row and each column. In Lean this is written as follows.

```
1 def magic_square_layout : LCSLayout := {
2   r := 6
3   s := 9
4   V := fun i =>
5     match i with
6     | 0 => {0, 1, 2}
7     | 1 => {3, 4, 5}
8     | 2 => {6, 7, 8}
9     | 3 => {0, 3, 6}
10    | 4 => {1, 4, 7}
11    | 5 => {2, 5, 8}
12 }
```

The corresponding game is obtained by specifying the right-hand side bits. Only the final column has odd parity, so only the last equation is assigned the value 1.

```
1 def magic_square_game : LCSGame
2   magic_square_layout := {
3     b := fun i => if i = {5, by decide} then 1 else 0
4   }
```

This illustrates the convenience of the support-based description. At this stage one only needs to specify the incidence pattern of the variables and the parity bit attached to each constraint. The resulting object is already enough to state the game and to instantiate the general strategy and winning-condition framework.

4.4 Formalising the Grid Strategy

The observable grid itself is encoded as a function from the nine variable indices to 4×4 matrices, obtained as Kronecker products of the Pauli matrices and the identity.

```
1 def magic_square_grid : Fin 9 → mat4
2   | 0 => X ⊗ I2
3   | 1 => I2 ⊗ X
4   | 2 => X ⊗ X
5   | 3 => I2 ⊗ Y
6   | 4 => Y ⊗ I2
7   | 5 => Y ⊗ Y
8   | 6 => X ⊗ Y
9   | 7 => Y ⊗ X
10  | 8 => Z ⊗ Z
```

To turn this grid into a strategy, one must verify that it satisfies the structural conditions required by the observable formalism. The first condition is that each grid entry is an observable, that is, a self-adjoint involution. This is proved in Lean by reducing each case to the corresponding facts for the Pauli matrices and using the compatibility of the Kronecker product with these properties.

The second condition is that, for every equation of the game, the observables lying in that equation commute pairwise. For the magic square this means proving pairwise commutation along each row and each column. The proof uses the familiar Pauli commutation and anticommutation relations, packaged through elementary lemmas about Kronecker products. Once these six commutativity checks are established, they are assembled into a uniform statement saying that the observables associated with any equation of the layout commute pairwise.

The resulting theorem-level object is the strategy

```
1 noncomputable def Strat_merminPeres :
2   BipartiteObservableStrategy (Fin 2 × Fin 2)
3   magic_square_layout where
4     obs := magic_square_grid
5     is_observable := magic_square_is_observable
6     sameEquation_comm := MP_sameEquation_comm
```

Notably, the case study not only defines the standard magic square strategy, but also formally verifies that the observable grid satisfies all algebraic hypotheses required by the abstract formalism. This makes the magic square a genuine verified example of the general definitions introduced earlier.

4.5 From the Game to the Associated Linear System

Although the game is described initially in support form, the later group-theoretic part of the development works with an explicit binary linear system. For this reason, the support-based game is converted to a `LinearSystem` using the generic map introduced in `LCSGame.toLinearSystem`.

For the magic square this yields the explicit system whose coefficient matrix has one row for each row or column support, and whose right-hand side vector records the parity pattern. In the codebase this is defined by

```
1 def magic_square_system : LinearSystem :=
2   magic_square_game.toLinearSystem
```

Thus the same concrete example appears in two complementary forms:

- as an `LCSGame`, convenient for the strategy and winning-condition constructions,
- as a `LinearSystem`, convenient for the solution-group construction.

4.6 The Solution Group for the Magic Square

Once the explicit binary linear system has been recovered, the generic solution-group construction can be instantiated directly. The magic-square solution group is simply the solution group attached to `magic_square_system`.

```
1 abbrev MPSolutionGroup := SolutionGroup
   magic_square_system
```

The formalisation then extracts inspectable data from this system: the coefficient rows, the right-hand side vector, the supports of the equations, and the resulting list of relators in presentation form. This does not yet prove any new analytic property of the concrete strategy, but it shows that the abstract algebraic machinery developed earlier can be applied to a canonical and highly nontrivial example.

In this sense, the magic square plays two roles in the project. First, it provides a concrete observable strategy whose validity can be checked directly in Lean. Second, it provides a concrete binary linear system and hence a concrete solution group to which the general representation-theoretic constructions apply.

4.7 Achievements of the Case Study

The present formalisation of the Mermin-Peres game therefore establishes the following points.

- The game itself is encoded explicitly as a binary LCS game.
- The standard Pauli-grid construction is encoded explicitly as a family of observables.
- Lean verifies that this grid satisfies the observable conditions and the same-equation commutation requirements needed to define a valid strategy.
- The support-based game is converted to an explicit binary linear system.

- The associated solution group is instantiated and made inspectable in the concrete magic-square case.

4.7.1 Incomplete Aspects of the Case Study

The case study lacks a theorem stating that this particular concrete strategy is a perfect strategy for the magic square game.

More precisely, the project contains a general result showing that if a suitable strategy annihilates the EPR state through the local loss operators, then one can extract row identities and construct a representation of the associated solution group. However, this EPR-annihilation hypothesis is not proved for the present concrete packaging of the magic-square strategy. Thus the case study currently verifies the strategy data and the associated algebraic structures, while stopping short of a complete formal proof of perfect play for this concrete example.

Even with this limitation, the magic square remains a useful and informative case study. It demonstrates that the abstract framework developed in the project is expressive enough to capture the most familiar example of quantum pseudotelepathy, and it provides a concrete benchmark against which the strategy, EPR, and solution-group layers of the formalisation can be understood.

5 Limitations and Future Work

5.1 Limitations

The present development has several important limitations.

- **Binary setting only.**

The whole development is restricted to LCS games over F_2 , and this choice is built in from the beginning through the support-based description of equations, which records only which variables appear and not general coefficients over an arbitrary field.

- **Finite-dimensional matrix setting for the EPR argument.**

The extraction results are proved only after specialising to complex matrices, so the final EPR and representation arguments do not yet apply in a more abstract operator-algebraic or infinite-dimensional setting.

- **No full equivalence theorem between the two strategy formalisms.**

The project constructs and uses the bridge from observable strategies to projector strategies, but it does not prove a complete round-trip equivalence showing that the two formalisms determine the same data in a canonical way. One concrete obstruction is that the observables recovered from a projector strategy on Alice's side are naturally indexed by a pair (i, j) of an equation and a variable in that equation, whereas `ObservableStrategy` is formulated with a single observable for each variable.

- **The current bipartite observable interface is too specialised for the full EPR converse story.**

The `BipartiteObservableStrategy` wrapper used in the EPR part of the development starts from a single family of observables and lifts it symmetrically to the two tensor factors. This is sufficient for packaging valid bipartite strategies and for the conditional row-identity and representation results proved in the project. However, the EPR extraction argument naturally produces transpose-related local observables, and the current interface does not yet formalise the more general two-family setup needed to reconstruct perfect strategies from solution-group representations or to instantiate the full perfect-play pipeline for the concrete Magic Square strategy.

- **No direct computation with real or complex operator entries.**

In Lean, real numbers are implemented in a proof-oriented way rather than as an efficient executable numeric type, so matrices with real or complex entries are well suited for exact reasoning but not for effective computation of concrete operator values.

5.2 Future Work

The project can be extended by addressing the limitations and/or adding more concrete examples. For instance, extending the formalisation to LCS games over arbitrary finite fields would be a natural next step, as well as completing the perfect-play proof for the magic square strategy.

More interestingly, the project could move toward **robust self-testing** results [4], by replacing the exact annihilation condition on the EPR state with an approximate version, and showing that this implies approximate versions of the row identities, which in turn can be used to show that the strategy is close to an ideal strategy in a suitable sense. This would require developing a robust version of the EPR extraction argument, which is a significant technical challenge but would be a very interesting direction for future work.

6 Conclusion

Overall, this project successfully formalises the core mathematical framework of binary Linear Constraint System games in Lean 4. By providing these foundational definitions and formalising several key results, including quantum strategy frameworks and matrix representations of the solution group, this work opens the door to verifying more advanced LCS game theory theorems in Lean.

7 AI Use Disclosure

The focus of this project is on the high-level formalisation architecture and theorem statements rather than the cleanliness of the proof bodies. Accordingly, the detailed Lean 4 proof steps and their execution were left to an AI coding assistant.

References

- [1] A. Einstein, B. Podolsky, and N. Rosen, “Can Quantum-Mechanical Description of Physical Reality Be Considered Complete?,” *Physical Review*, vol. 47, no. 10, pp. 777–780, 1935, doi: [10.1103/PhysRev.47.777](https://doi.org/10.1103/PhysRev.47.777).
- [2] J. S. Bell, “On the Einstein Podolsky Rosen Paradox,” *Physica Physique Fizika*, vol. 1, no. 3, pp. 195–200, 1964, doi: [10.1103/PhysicsPhysique-Fizika.1.195](https://doi.org/10.1103/PhysicsPhysique-Fizika.1.195).
- [3] R. Cleve and R. Mittal, “Characterization of Binary Constraint System Games.” [Online]. Available: <https://arxiv.org/abs/1209.2729>
- [4] A. Coladangelo and J. Stark, “Robust self-testing for linear constraint system games.” [Online]. Available: <https://arxiv.org/abs/1709.09267>
- [5] A. Mehta, “Entanglement and Non-locality in Games and Graphs,” Doctoral dissertation, 2021. [Online]. Available: <http://hdl.handle.net/1807/106342>
- [6] N. D. Mermin, “Simple Unified Form for the Major No-Hidden-Variables Theorems,” *Physical Review Letters*, vol. 65, no. 27, pp. 3373–3376, 1990, doi: [10.1103/PhysRevLett.65.3373](https://doi.org/10.1103/PhysRevLett.65.3373).